

Mobile Robotics Lab Manual

Lab Session 6: Reactive Navigation Using Turtlebot3 LiDAR in Gazebo Environment

Azmeer Ali
University of Engineering and Technology (UET) Lahore

May 10, 2026

1 Objective

The primary objective of this lab session was to implement reactive navigation utilizing LiDAR data within a Gazebo simulated environment. This involved processing the `/scan` topic to perceive the robot's surroundings and executing fundamental autonomous behaviors, including obstacle detection, obstacle avoidance, and wall following, without relying on pre-generated maps.

2 Methodology and Execution

2.1 Environment Setup

The simulation was initialized by launching the TurtleBot3 `burger` model within the standard Gazebo world using the `turtlebot3_gazebo` package. The environment provided various structural obstacles to test the reactive navigation logic.

2.2 Scan Processing and Region Extraction

The raw LiDAR data structure was analyzed to extract meaningful directional distances. The 360-degree `LaserScan` array was filtered to remove invalid readings (such as `inf`, `NaN`, and false zeros) by capping out-of-range values to the sensor's maximum reliable range (3.5 meters). The cleaned data was then sliced into three primary regions: **Front** (-30° to $+30^\circ$), **Left** (30° to 90°), and **Right** (270° to 330°). The minimum distance within each region was computed to determine the proximity of the nearest obstacles.

2.3 Reactive Behavior Implementation

A custom ROS 2 node, `lidar_navigator`, was developed to sequence three distinct behaviors based on the processed scan data:

- **Stop-on-Obstacle:** A safety threshold was established (0.6 meters). If the minimum front distance dropped below this threshold, the linear velocity was immediately set to 0.0 m/s.

- **Obstacle Avoidance:** Upon halting for a front obstacle, the logic compared the left and right clearances. The robot published an angular velocity to turn toward the direction with the larger clearance space.
- **Wall Following:** While moving forward in unobstructed space, a proportional controller was applied using the lateral (left and right) scan distances. The error between the left and right distances was multiplied by a proportional gain ($K_p = 0.5$) to generate corrective angular velocities, keeping the robot centered within corridors.

3 ROS 2 Node Implementation

```

1 import rclpy
2 from rclpy.node import Node
3 from sensor_msgs.msg import LaserScan
4 from geometry_msgs.msg import Twist
5 import numpy as np
6
7 class LidarNavigator(Node):
8     def __init__(self):
9         super().__init__('lidar_navigator')
10        self.subscription = self.create_subscription(
11            LaserScan,
12            '/scan',
13            self.scan_callback,
14            10)
15        self.publisher = self.create_publisher(Twist, '/cmd_vel', 10)
16
17        self.front_threshold = 0.6 # Stop if obstacle is within 0.6
18        self.side_threshold = 0.4 # Threshold for side clearance
19
20    def scan_callback(self, msg):
21        ranges = np.array(msg.ranges)
22
23        # Clean data (remove inf/nan and false zeros)
24        ranges = np.where(np.isinf(ranges), 3.5, ranges)
25        ranges = np.where(np.isnan(ranges), 3.5, ranges)
26        ranges = np.where(ranges == 0.0, 3.5, ranges)
27
28        # Define regions
29        front = np.concatenate((ranges[0:30], ranges[330:360]))
30        left = ranges[30:90]
31        right = ranges[270:330]
32
33        # Compute minimum distance in each region
34        front_dist = np.min(front)
35        left_dist = np.min(left)
36        right_dist = np.min(right)
37
38        twist = Twist()
39
40        # Obstacle logic and Behavior Sequencing
41        if front_dist < self.front_threshold:
42            # Stop-on-obstacle behavior

```

```

43         self.get_logger().info(f"Obstacle Ahead! Distance: {
front_dist:.2f}m")
44         twist.linear.x = 0.0
45
46         # Turn direction logic
47         if left_dist > right_dist:
48             twist.angular.z = 0.5    # Turn Left
49         else:
50             twist.angular.z = -0.5   # Turn Right
51     else:
52         # Forward motion & Proportional Wall Following
53         twist.linear.x = 0.15
54
55         if left_dist < 1.0 and right_dist < 1.0:
56             error = left_dist - right_dist
57             twist.angular.z = 0.5 * error
58         else:
59             twist.angular.z = 0.0
60
61         self.publisher.publish(twist)
62
63 def main(args=None):
64     rclpy.init(args=args)
65     node = LidarNavigator()
66     rclpy.spin(node)
67     node.destroy_node()
68     rclpy.shutdown()
69
70 if __name__ == '__main__':
71     main()

```

Listing 1: LiDAR Navigator Node (lidar_navigator.py)

4 Conclusion

This lab session effectively demonstrated the principles of reactive navigation by translating raw sensor data into immediate physical action. By dividing the `LaserScan` array into distinct spatial regions, the robot successfully navigated a complex environment autonomously. The implementation emphasized the critical nature of sensor data sanitization, specifically handling infinite or null values returned by the LiDAR. Furthermore, tuning the proportional control gain and threshold parameters highlighted the balance required between responsive obstacle avoidance and stable wall-following dynamics, establishing a robust foundation for mapless autonomous navigation.